

# GUI64

## API Documentation

Programmer's Reference for GUI64 Applications



Public API base:     \$5F00  
Application space:   \$B000-\$CFFF  
CPU:                 NMOS 6502  
Assembler syntax:    ACME  
API entries:          33

## Contents

---

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Introductory Notes</b>                    | <b>2</b>  |
| <b>2</b> | <b>Creating Windows, Controls, and Menus</b> | <b>2</b>  |
| 2.1      | Windows . . . . .                            | 2         |
| 2.2      | Controls . . . . .                           | 3         |
| 2.3      | Menus . . . . .                              | 5         |
| <b>3</b> | <b>Character set</b>                         | <b>5</b>  |
| <b>4</b> | <b>Two Designs</b>                           | <b>6</b>  |
| <b>5</b> | <b>Public Jump Table</b>                     | <b>6</b>  |
| 5.1      | Window API . . . . .                         | 7         |
| 5.2      | Control API . . . . .                        | 8         |
| 5.3      | Application Extensions . . . . .             | 10        |
| 5.4      | Timer API . . . . .                          | 11        |
| 5.5      | Menu, Mouse and Design Helpers . . . . .     | 11        |
| 5.6      | Low-level Buffer Helpers . . . . .           | 12        |
| <b>6</b> | <b>Hello World Application</b>               | <b>12</b> |

## 1 Introductory Notes

GUI64 exposes its application interface through a sequence of three-byte JMP entries beginning at **\$5F00**. These routines are given in Section 5 alongside with their particular explanation. However, a programmer should always use the most recent version of the file

**gui64.inc.asm**

which contains these addresses and many other constants definitions.

The current GUI64 application area is:

**\$B000 ... \$CFFF**

**Warning!** GUI64 uses zero-page memory extensively. In particular, temporary pointers such as **\$02/\$03**, **\$FB/\$FC**, and **\$FD/\$FE** must be treated as volatile across API calls unless a routine explicitly documents otherwise.

**Calling Convention:** Unless stated otherwise,

- 16-bit addresses are passed with **X** as low byte and **Y** as high byte (**Addr(X/Y)**).
- The current window is mirrored in the static window structure at **\$10-\$1F**.
- The current control is mirrored in the static control structure at **\$20-\$2F**.
- wndParam0/wndParam1 carry window-procedure event information.
- Carry is used by several yes/no-routines as the Boolean result.
- API routines may alter **A**, **X**, **Y**, flags and GUI64 scratch memory unless documented otherwise.

## 2 Creating Windows, Controls, and Menus

### 2.1 Windows

A GUI64 window in C64 memory is defined by a 16 byte structure containing the following properties (the “Reserved”-entries are used by GUI64 internally and should not be modified by an app):

| Offset | Name           | Meaning                               |
|--------|----------------|---------------------------------------|
| 0      | CurrentWindow  | Window identifier handle/index        |
| 1      | WindowType     | Window type (must be at least 50)     |
| 2      | Reserved       | -                                     |
| 3      | WindowBits     | Window property flags                 |
| 4      | WindowPosX     | X position                            |
| 5      | WindowPosY     | Y position                            |
| 6      | WindowWidth    | Width                                 |
| 7      | WindowHeight   | Height                                |
| 8      | WindowTitleStr | 16-bit pointer to string in title bar |
| 10     | Reserved       | -                                     |
| 12     | Reserved       | -                                     |
| 13     | Reserved       | -                                     |
| 14     | WindowProc     | 16-bit window-procedure pointer       |

**Window bits:** The window bits are (see gui64.inc.asm) **BIT\_WND\_HASMENU** (1), **BIT\_WND\_RESIZABLE** (2), **BIT\_WND\_FIXEDWIDTH** (4), **BIT\_WND\_FIXEDHEIGHT** (8), **BIT\_WND\_CANMAXIMIZE** (16) (WIN only), **BIT\_WND\_CANMINIMIZE** (32) (WIN only), **BIT\_WND\_ISMINIMIZED** (64), **BIT\_WND\_ISMAXIMIZED** (128)(WIN only)

**Window proc:** An important part of a window is its window proc. This routine handles user input events like mouse/joystick or keyboard actions. When the program pointer enters the window proc, the event code can be read from **wndParam0** (see gui64.inc.asm). The events are (see gui64.inc.asm):

**EC\_RBTNPRESS**, **EC\_RBTNRELEASE**, **EC\_LBTNPRESS**, **EC\_LBTNRELEASE**, **EC\_MOUSEMOVE**, **EC\_DBLCLICK**, **EC\_SCROLLWHEELDOWN**, **EC\_SCROLLWHEELUP**, and **EC\_KEYPRESS**.

The other parameter **wndParam1** contains the **program mode** that was active when the event was triggered. The program modes are (see gui64.inc.asm) **PM\_NORMAL** (0), **PM\_MENU** (1), and **PM\_DIALOG** (255). For example, when the user selects a menu item of the window's menu, the window proc receives an event with **EC\_LBTNPRESS** in **wndParam0** and **PM\_MENU** in **wndParam1**. The program mode can be read at any time from **ProgramMode** (see gui64.inc.asm).

**Warning:** In each window proc, apps should call **GUI\_StdWndProc** which handles events sent to the GUI64 standard controls. For example, when the user clicks on a button, the standard window proc forwards the event to the button's action routine.

**Creation:** A window is created by either using **GUI\_CreateWindow** or **GUI\_CreateWindowEx**. While the first just creates a window and hence only expects window data, the second creates a window with controls. Therefore, there must be controls data provided right after the window data. Window data must always be provided in the following form:

- window type identifier (value must be at least 50), bits, xPos, yPos, width, height, 16-bit pointer to title string, 16-bit pointer to window proc

Each of the first 6 entries is a byte. The last two are words (pointers in little endian format).

## 2.2 Controls

Controls are graphical elements in a window. There are 13 standard controls in GUI64 that an app may use. Some of them are dynamic (action) and react to user input, some are static (no action). The list of controls is:

| ID | Type identifier     | Control            | Action |
|----|---------------------|--------------------|--------|
| 1  | CT_MENUBAR          | Menu bar           | Yes    |
| 2  | CT_BUTTON           | Button             | Yes    |
| 3  | CT_LISTBOX          | List box           | Yes    |
| 5  | CT_COLORPICKER      | Color picker       | Yes    |
| 6  | CT_RADIOBUTTONGROUP | Radio button group | Yes    |
| 7  | CT_UPDOWN           | Up/down control    | Yes    |
| 8  | CT_EDIT_SL          | Single-line edit   | Yes    |
| 10 | CT_CHECKBOX         | Check box          | Yes    |
| 31 | CT_LABEL            | Label              | No     |
| 32 | CT_LABEL_ML         | Multi-line label   | No     |
| 33 | CT_FRAME            | Frame              | No     |
| 34 | CT_PROGRESSBAR      | Progress bar       | No     |
| 35 | CT_COLBOXLABEL      | Color box label    | No     |

The control type identifier is just a replacement of the ID (e.g., **CT\_MENUBAR** = 1).

Each control in a window has properties, provided by a 16 byte structure in memory. Based on this structure, GUI64 knows how to paint the control. The entries of this structure are:

| Offset | Name         | Meaning                            |
|--------|--------------|------------------------------------|
| 0      | ControlIndex | Control index                      |
| 1      | ControlType  | Control type (must be at least 50) |
| 2      | ControlColor | Color                              |
| 3      | ControlPosX  | X position                         |

| Offset | Name            | Meaning                           |
|--------|-----------------|-----------------------------------|
| 4      | ControlPosY     | Y position                        |
| 5      | ControlWidth    | Width                             |
| 6      | ControlHeight   | Height                            |
| 7      | ControlBits     | Control property flags            |
| 8      | ControlHilIndex | Highlighted item index            |
| 9      | ControlTopIndex | Top item / control-specific value |
| 10     | ControlNumStr   | Number of strings                 |
| 11     | ControlStrings  | 16-bit string/string-list pointer |
| 13     | -               | Control-specific                  |
| 14     | ControlBitsEx   | Extended control flags            |
| 15     | -               | Control-specific                  |

For some controls, the struct members 9 - 14 are used for something else:

| Offset | Name              | Meaning                                           |
|--------|-------------------|---------------------------------------------------|
| 10     | UpDown_LowerLimit | Lower limit of UpDown value                       |
| 11     | UpDown_UpperLimit | Upper limit of UpDown value                       |
| 12     | UpDown_DigitLo    | Low digit of UpDown control                       |
| 13     | UpDown_DigitHi    | Hi digit of UpDown control                        |
| 10     | ProgressBar_MaxLo | Low byte of maximal value of pogress bar          |
| 11     | ProgressBar_MaxHi | High byte of maximal value of pogress bar         |
| 12     | ProgressBar_ValLo | Low byte of value of pogress bar                  |
| 13     | ProgressBar_ValHi | Low byte of value of pogress bar                  |
| 9      | EditSL_CaretPos   | Position of caret in single-line edit control     |
| 10     | EditSL_MaxStrLen  | Maximal string length in single-line edit control |
| 14     | EditSL_Forbidden  | 16-bit pointer to forbidden chars                 |

**Selection and modification:** In GUI64, windows and controls are *selected*. This means that window or control structures are copied from the heap to zero page (**\$10 - \$1f** for a window and **\$20 - \$2f** for a control). Once selected, a window or control is the *current* window or control. You select windows and controls by using [GUI\\_SelectWindow](#) or [GUI\\_SelectControl](#), respectively. When you have selected a window or control and modify its structure in zero page, you have to use [GUI\\_UpdateWindow](#) or [GUI\\_UpdateControl](#) to have the changes take effect.

**Creation:** There are two options to create a control. The first is using [GUI\\_CreateWindowEx](#), where a window is created alongside its controls. Static control information data must always be provided in the following form:

- control type identifier (value must be at least 50), xPos, yPos, width, height, null terminated string

Each entry is a byte. The null-terminated string consists of chars (Petscii) and might be empty (i.e., 0).

When you use [GUI\\_CreateWindowEx](#) for creating the controls in your window, you provide the window information data at **Addr(X/Y)**, followed by the list of controls information data. Don't forget to add a terminating zero at the end of the list of controls, so GUI64 knows that this was the last control in the list.

The second option to create a control is to use [GUI\\_AddControl](#). This adds a control with control information of the above form at **Addr(X/Y)** to the current window.

**Custom controls:** Programmers may register new control types. Custom control type identifiers start at 50. The new controls are added as described above. But the programmer has to tell GUI64 before that they want to add custom controls. For this, two routines have to be provided:

- one routine for painting the new controls
- one routine for defining the default behavior of the new controls (event reaction)

The programmer registers these two routines via [GUI\\_SetPaintCtrlsRoutine](#) and [GUI\\_SetCtrlActionsRoutine](#), respectively. Once the program counter enters these two routines, the **X** register is set as the control type identifier. A

control is painted by storing character indices in `$FD/$FE (sta ($fd),y)` and color indices in `$02/$03 (sta ($02),y)`. To get to the next row, one may make use of the API calls `GUI_AddBufWidthToFD` and `GUI_AddBufWidthTo02`, respectively. In the `CtrlActions` routine, you may use `wndParam0` and `wndParam1` in the same way as in the window proc (see above). If you only use one instance of a custom control, you may alternatively define the control's event behavior in the parent windows's window proc.

## 2.3 Menus

In GUI64, any window may possess a menubar. This is indicated in the window bits (`BIT_WND_HASMENU`). In the WIN version of GUI64, the menubar is placed inside the window right below the title bar, while it is placed in the system bar at the top of the screen in the MAC version. Therefore, if a window possesses a menubar, its height in the WIN version should be one unit larger than in the MAC version to display the same content. The programmer can take care of this behavior by using the API call `GUI_GetDesign`.

A menubar is in fact a control whose control index should always be zero. As its position is not relevant, the `xPos` and `yPos` values in the control information data are used as a pointer to a list of 16-bit pointers to drop-down menus. Each drop-down menu is defined as a structure of the following form:

- menu ID, maximal string length of menu items, number of menu items, list of null-terminated strings

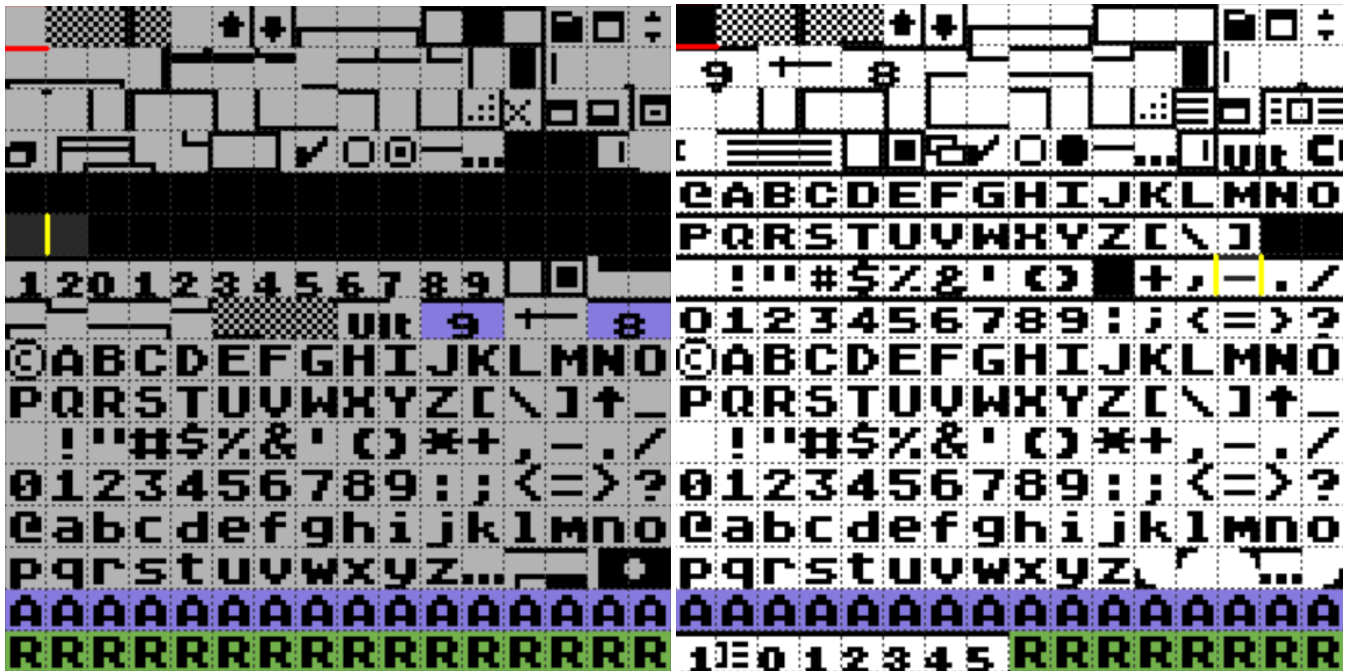
Menu IDs start at **10**.

Here is an example from the demo app *Sweepy*:

```
; Constants
MENU_MS_GAME = 10
MENU_MS_HELP = 11
...
...
...
; Control definition
!byte CT_MENUBAR, <MSMenubar, >MSMenubar, 0, 0
!pet 0
...
...
...
; Definition of menu bar
MSMenubar      !word Menu_MS_Game, Menu_MS_Help
Str_MSMenubar  !pet "Game",0,"?",0
; Definition of menus
; Format: ID, max_str_len, item_count, Stringlist
Menu_MS_Game   !pet ID_MENU_GAME,4,2,"New",0,"Quit",0
Menu_MS_Help   !pet ID_MENU_HELP,5,1,"About",0
```

## 3 Character set

As most C64 software using character graphics mode, GUI64 uses a custom character set. The character sets of WIN and MAC design look as follows:



The 16 blue A's (indices 224 - 239) are at the same place in both designs. This area is reserved for custom characters defined by an app. For defining new characters for your app, use the API call [GUI\\_RegisterChars](#).

## 4 Two Designs

As there exist two designs of GUI64 (WIN version and MAC version), the programmer should always check whether the controls are placed as intended in both designs. For this, they can use [GUI\\_GetDesign](#).

## 5 Public Jump Table

| Address | Entry point                            | Group   | Purpose                                                                   |
|---------|----------------------------------------|---------|---------------------------------------------------------------------------|
| \$5F00  | <a href="#">GUI_CreateWindow</a>       | Window  | Create a window from a static window definition                           |
| \$5F03  | <a href="#">GUI_CreateWindowEx</a>     | Window  | Create a window including the controls that follow the window definition  |
| \$5F06  | <a href="#">GUI_KillCurWindow</a>      | Window  | Remove the current window and its controls                                |
| \$5F09  | <a href="#">GUI_StdWndProc</a>         | Events  | Run GUI64 standard window event processing                                |
| \$5F0C  | <a href="#">GUI_IsInCurControl</a>     | Control | Test whether the mouse position is inside the current control             |
| \$5F0F  | <a href="#">GUI_Repaint</a>            | Paint   | Repaint the GUI                                                           |
| \$5F12  | <a href="#">GUI_GetDesign</a>          | System  | Return the active GUI64 design variant (WIN/MAC)                          |
| \$5F15  | <a href="#">GUI_UpdateControl</a>      | Control | Write the current static control structure back to the control heap       |
| \$5F18  | <a href="#">GUI_SelectCtrl_AddBits</a> | Control | MAC helper for selected-control flags; does nothing in the WIN build      |
| \$5F1B  | <a href="#">GUI_SetCtrlStringList</a>  | Control | Assign a string-list pointer and number of strings to the current control |
| \$5F1E  | <a href="#">GUI_SetCtrlString</a>      | Control | Assign a string pointer to the current control                            |
| \$5F21  | <a href="#">GUI_SelectControl</a>      | Control | Select a control by its index                                             |
| \$5F24  | <a href="#">GUI_SelectControl0</a>     | Control | Select control with index 0                                               |
| \$5F27  | <a href="#">GUI_SelectControl1</a>     | Control | Select control with index 1                                               |
| \$5F2A  | <a href="#">GUI_SelectControl2</a>     | Control | Select control with index 2                                               |
| \$5F2D  | <a href="#">GUI_SelectControl3</a>     | Control | Select control with index 3                                               |
| \$5F30  | <a href="#">GUI_SelectControl4</a>     | Control | Select control with index 4                                               |

| Address | Entry point                               | Group     | Purpose                                                                                                |
|---------|-------------------------------------------|-----------|--------------------------------------------------------------------------------------------------------|
| \$5F33  | <a href="#">GUI_UpdateWindow</a>          | Window    | Write the current static window structure back to the window heap                                      |
| \$5F36  | <a href="#">GUI_SetEditSLInfo</a>         | Control   | Set single-line edit control information                                                               |
| \$5F39  | <a href="#">GUI_RegisterChars</a>         | App       | Register up to 16 application-defined characters                                                       |
| \$5F3C  | <a href="#">GUI_SetCtrlActionsRoutine</a> | App       | Register the application control-event routine for newly defined controls                              |
| \$5F3F  | <a href="#">GUI_SetPaintCtrlsRoutine</a>  | App       | Register the application control-paint routine for newly defined controls                              |
| \$5F42  | <a href="#">GUI_AddBufWidthToFD</a>       | Low-level | Move paint cursor in <b>\$FD/\$FE</b> down by 1                                                        |
| \$5F45  | <a href="#">GUI_AddBufWidthTo02</a>       | Low-level | Move color cursor in <b>\$02/\$03</b> down by 1                                                        |
| \$5F48  | <a href="#">GUI_IsInCurMenu</a>           | Menu      | Test whether the mouse position is inside the current menu                                             |
| \$5F4B  | <a href="#">GUI_GetCurMenuID</a>          | Menu      | Return the current menu ID                                                                             |
| \$5F4E  | <a href="#">GUI_ShowMessage</a>           | Dialog    | Show a message dialog box                                                                              |
| \$5F51  | <a href="#">GUI_GetMousePosInWnd</a>      | Mouse     | Update <a href="#">MousePosInWndX/Y</a> with coordinates relative to the current window's control area |
| \$5F54  | <a href="#">GUI_GetCSTMWindowColor</a>    | System    | Return the custom window color                                                                         |
| \$5F57  | <a href="#">GUI_GetCSTMDesktopColor</a>   | System    | Return the custom desktop color                                                                        |
| \$5F5A  | <a href="#">GUI_StartTimer</a>            | Timer     | Start the application timer; ticks are generated every 1/10 second                                     |
| \$5F5D  | <a href="#">GUI_StopTimer</a>             | Timer     | Stop the application timer                                                                             |
| \$5F60  | <a href="#">GUI_InitTimer</a>             | Timer     | Register the application timer handler address                                                         |

## 5.1 Window API

### GUI\_CreateWindow \$5F00

Creates a window without controls.

#### INPUT

| Reg.     | Value        | Meaning                                         |
|----------|--------------|-------------------------------------------------|
| <b>X</b> | address low  | Low byte of the static window-definition table  |
| <b>Y</b> | address high | High byte of the static window-definition table |

The window-definition table should contain the bytes: window type, bits, xPos, yPos, width, height, pointer to title string, and pointer to window proc.

```
ldx #<Wnd_Data
ldy #>Wnd_Data
jsr GUI_CreateWindow
...
Wnd_Data !byte WT_MyWnd, %00100001,11,2,11,15, <Str_Title, >Str_Title, <MyWndProc, >MyWndProc
```

### GUI\_CreateWindowEx \$5F03

Creates a window with controls.

#### INPUT

| Reg.     | Value        | Meaning                                         |
|----------|--------------|-------------------------------------------------|
| <b>X</b> | address low  | Low byte of the static window-definition table  |
| <b>Y</b> | address high | High byte of the static window-definition table |

The window-definition table should contain the bytes: window type, bits, xPos, yPos, width, height, pointer to title string, and pointer to window proc, followed by the static control data and a termination zero byte.

```
ldx #<Wnd_Data
ldy #>Wnd_Data
jsr GUI_CreateWindowEx
...
Wnd_Data !byte WT_MyWnd, %00100001,11,2,11,15, <Str_Title, >Str_Title, <MyWndProc, >MyWndProc
; Controls list
!byte CT_MENUBAR, <Menubar, >Menubar, 0, 0
!pet 0
!byte CT_LABEL, 1, 1, 11, 1
!pet "Hello world",0
!byte CT_BUTTON, 6, 0, 5, 3
!pet " OK ",0
; termination zero byte
!byte 0
```

#### GUI\_KillCurWindow \$5F06

Removes the current window and its controls from window and control heap.

**Warning:** This call uses multiple GUI64 zero-page scratch locations. Do not keep an application pointer in temporary GUI64 zero-page storage across this call.

#### GUI\_StdWndProc \$5F09

Runs the standard GUI64 window proc: updates the mouse position relative to the current window and then dispatches standard behavior for mouse, joystick and keyboard events.

This routine should be called in each window proc.

```
MyWndProc jsr GUI_StdWndProc
; application-specific processing follows
rts
```

#### GUI\_UpdateWindow \$5F33

Copies the 16-byte static current-window structure in **\$10 - \$1f** to the window heap.

```
lda #20
sta WindowWidth
jsr GUI_UpdateWindow
```

## 5.2 Control API

#### GUI\_IsInCurControl \$5F0C

Tests whether MousePosInWndX/Y lies inside the current control.

**OUTPUT**

---

| Result      | Meaning                              |
|-------------|--------------------------------------|
| Carry set   | Mouse is inside the current control  |
| Carry clear | Mouse is outside the current control |

**GUI\_UpdateControl \$5F15**

Copies the 16-byte static current-control structure in **\$20 - \$2f** to the control heap.

```
lda #CL_WHITE
sta ControlColor
jsr GUI_UpdateControl
```

**GUI\_SetCtrlStringList \$5F1B**

Assigns a string-list address and its item count to the current control, then updates the heap entry. Hence, no **GUI\_UpdateControl** call necessary.

**INPUT**

| Reg. | Value        | Meaning                        |
|------|--------------|--------------------------------|
| A    | count        | Number of strings in the list  |
| X    | address low  | String-list address, low byte  |
| Y    | address high | String-list address, high byte |

```
jsr GUI_SelectControl0
ldx #<Str_Menu
ldy #>Str_Menu
lda #2
jsr GUI_SetCtrlStringList
```

**GUI\_SetCtrlString \$5F1E**

Assigns a string address to the current control and updates the heap entry. Hence, no **GUI\_UpdateControl** call necessary.

**INPUT**

| Reg. | Value        | Meaning                   |
|------|--------------|---------------------------|
| X    | address low  | String address, low byte  |
| Y    | address high | String address, high byte |

```
ldx #<Str_Label
ldy #>Str_Label
jsr GUI_SetCtrlString
```

**GUI\_SelectControl \$5F21**

Selects a control by index and loads its heap entry into the static control structure in **\$20 - \$2f**.

**INPUT**

| Reg. | Value         |
|------|---------------|
| A    | Control index |

```
lda #5
jsr GUI_SelectControl
```

The convenience entries **GUI\_SelectControl0** through **GUI\_SelectControl4** select indices 0 through 4 without requiring the index in **A**.

## 5.3 Application Extensions

### GUI\_RegisterChars \$5F39

Copies application-defined character data into the application-character area in the GUI64 character set.

INPUT

| Reg. | Value        | Meaning                            |
|------|--------------|------------------------------------|
| A    | 1-16         | Number of characters               |
| X    | address low  | Character table address, low byte  |
| Y    | address high | Character table address, high byte |

Values of 17 or greater are rejected by returning without copying.

```
ldx #<CharList
ldy #>CharList
lda #14
jsr GUI_RegisterChars
```

### GUI\_SetCtrlActionsRoutine \$5F3C

Registers the application routine that handles events for application-defined controls.

INPUT

| Reg. | Value        | Meaning                    |
|------|--------------|----------------------------|
| X    | address low  | Handler address, low byte  |
| Y    | address high | Handler address, high byte |

Once the program pointer enters the provided actions routine, the **X** register is the control type.

```
ldx #<MyCtrlAction
ldy #>MyCtrlAction
jsr GUI_SetCtrlActionsRoutine
```

### GUI\_SetPaintCtrlsRoutine \$5F3F

Registers the application routine used to paint application-defined controls.

INPUT

| Reg. | Value | Meaning |
|------|-------|---------|
|------|-------|---------|

|          |              |                                  |
|----------|--------------|----------------------------------|
| <b>X</b> | address low  | Paint routine address, low byte  |
| <b>Y</b> | address high | Paint routine address, high byte |

Once the program pointer enters the provided paint routine, the **X** register is the control type. Moreover, **\$fd/\$fe** points to the position of the control in the paint buffer, and **\$02/\$03** points to the position in the color buffer.

```
ldx #<MyCtrlPaintRoutine
ldy #>MyCtrlPaintRoutine
jsr GUI_SetPaintCtrlsRoutine
```

## 5.4 Timer API

### GUI\_InitTimer \$5F60

Registers the application's timer handler.

#### INPUT

| Reg.     | Value        | Meaning                          |
|----------|--------------|----------------------------------|
| <b>X</b> | address low  | Timer-handler address, low byte  |
| <b>Y</b> | address high | Timer-handler address, high byte |

```
ldx #<MyTimerHandler
ldy #>MyTimerHandler
jsr GUI_InitTimer
```

### GUI\_StartTimer \$5F5A

Starts the application timer. GUI64 generates timer events at one-tenth-second granularity from the CIA TOD clock.

### GUI\_StopTimer \$5F5D

Stops the application timer.

## 5.5 Menu, Mouse and Design Helpers

### GUI\_GetCurMenuID \$5F4B

#### OUTPUT

| Reg.     | Value   |
|----------|---------|
| <b>A</b> | menu ID |

```
jsr GUI_GetCurMenuID
cmp #ID_MENU_GAME
beq GameMenuClicked
```

**GUI\_GetMousePosInWnd \$5F51**

Updates **MousePosInWndX** and **MousePosInWndY** with coordinates relative to the area of the current window in which controls are placed.

**GUI\_GetCSTMWindowColor \$5F54****OUTPUT**

| Reg. | Value                 |
|------|-----------------------|
| A    | Standard window color |

**GUI\_GetCSTMDesktopColor \$5F57****OUTPUT**

| Reg. | Value                  |
|------|------------------------|
| A    | Standard desktop color |

## 5.6 Low-level Buffer Helpers

The following public entries expose GUI64 pointer arithmetic. They are useful for custom controls, but unlike higher-level calls they operate on GUI64's scratch pointers directly.

| Entry point                | Operation                                                                    |
|----------------------------|------------------------------------------------------------------------------|
| <b>GUI_AddBufWidthToFD</b> | Move position <b>\$fd/\$fe</b> down by 1 in paint buffer (increment y coord) |
| <b>GUI_AddBufWidthTo02</b> | Move position <b>\$02/\$03</b> down by 1 in color buffer (increment y coord) |

**Warning:** These pointers are GUI64 working storage. Their contents are not preserved by unrelated GUI calls.

## 6 Hello World Application

The following application shows a window with a label displaying the text *Hello World*.

```
!source "gui64.inc.asm"

WT_MYWINDOW = 50

*=$b000

    ldx #<MyWindow
    ldy #>MyWindow
    jmp GUI_CreateWindowEx

MyWndProc    jmp GUI_StdWndProc

MyWindow     !byte WT_MYWINDOW,BIT_WND_CANMINIMIZE,5,5,15,10
             !word Str_Title,MyWndProc
             !byte CT_LABEL,1,1,11,1
             !pet "Hello World",0
             ; Termination zero byte
             !byte 0

Str_Title    !pet "Hi",0
```